

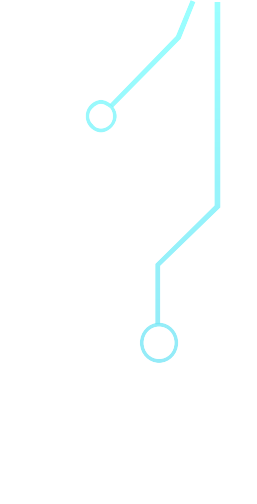
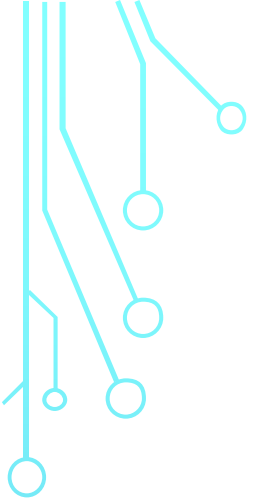
A decorative graphic on the left side of the slide, consisting of a network of thin, gold-colored lines and small circles, resembling a circuit board or a neural network diagram.

Quiz 1 Review

COMP211 Connor McMahon

Check Canvas for Quiz Info

- <https://uncch.instructure.com/courses/120642/pages/quiz-1>
- Here, you will find
 - Logistics
 - Review Session and Tutoring
 - Material to Review
 - Practice Problems
 - Learning Objectives



`.bashrc`

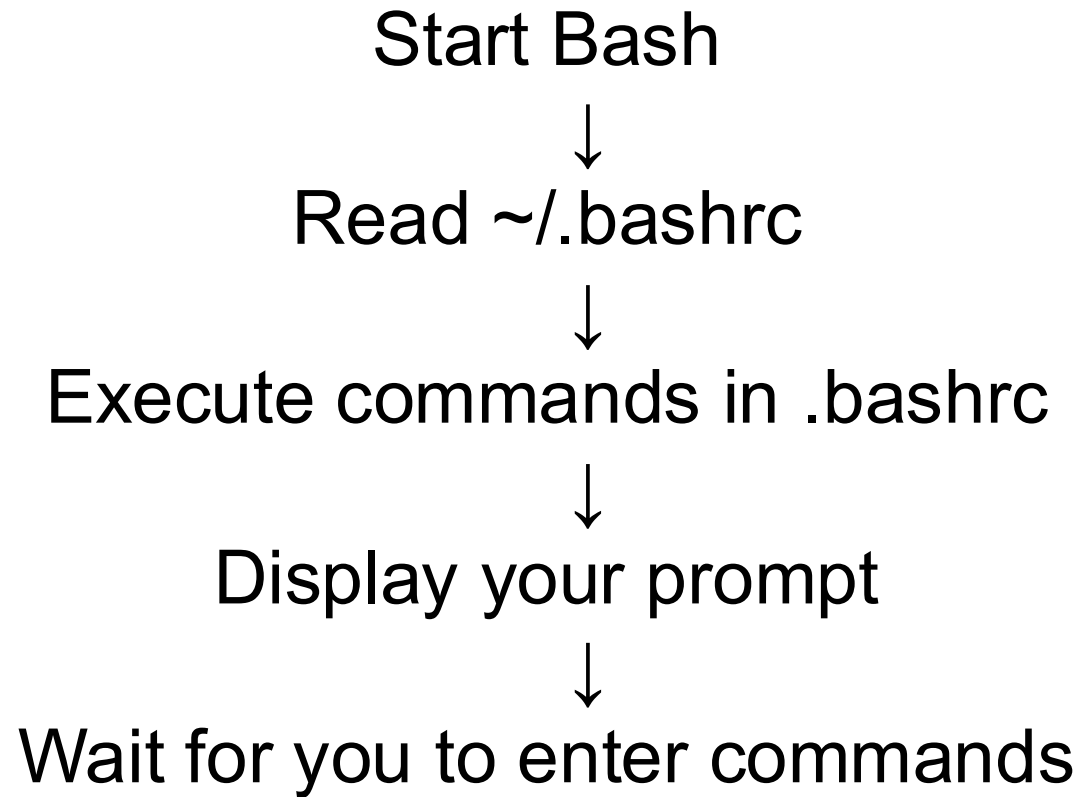
.bashrc

- Your **.bashrc** is a configuration file for Bash. It contains commands that Bash runs when you start a new interactive Bash shell.
- The file is located in your **home directory**:
 - `~/ .bashrc`
- The `.` at the beginning of the filename makes it a **hidden file**, so it normally won't appear with a plain `ls`. You can see it with:

```
$ ls -a ~
```

.bashrc

- When you open a terminal and Bash starts, the sequence is as follows:



.bashrc

- An important thing to remember is that `.bashrc` is not just a list of settings.
- It contains Bash commands. When Bash reads `.bashrc`, it executes those commands.
- If you modify `.bashrc`, the changes generally won't affect a shell that's already running. You can either open a new Bash shell or tell the current shell to read the file again:

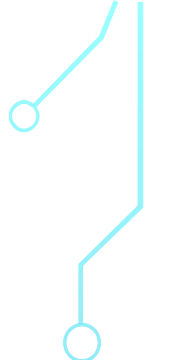
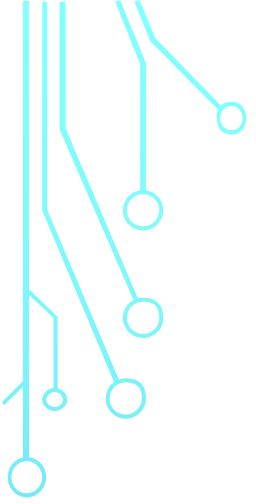
```
$ source ~/.bashrc
```
- The `source` command tells Bash to read a file and execute the commands in that file in the current shell.

source

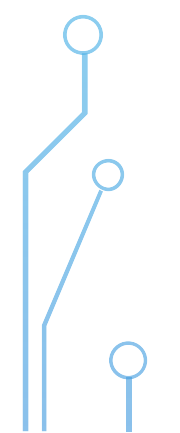
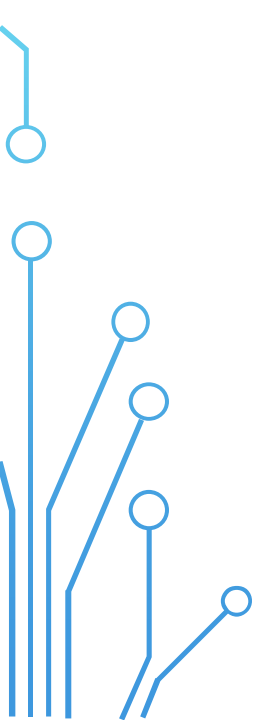
- You've probably seen source before
- When working with Python, you may have created a virtual environment:

```
python3 -m venv .venv
```
- and then activated it with:

```
source .venv/bin/activate
```
- The activate file is actually a **shell script** containing commands that modify your current shell.



Python Virtual Environments

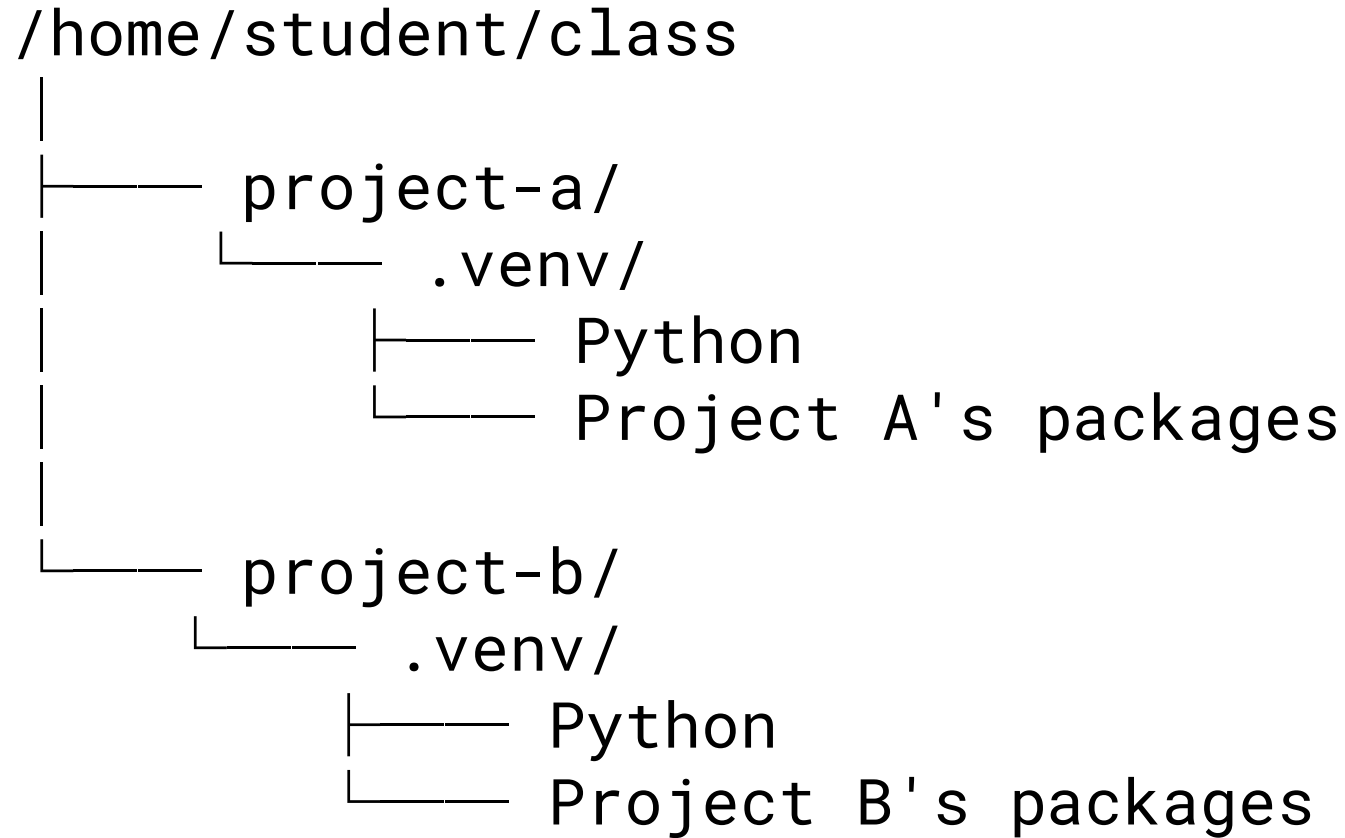


Python Virtual Environments

- A Python virtual environment is an isolated environment for a Python project that has its own Python executable and its own set of installed Python packages.
- Why do we use them? Imagine you have two projects:
 - Project A needs package version 1
 - Project B needs package version 2
- If every Python project uses the same globally installed packages, these requirements can conflict.

Python Virtual Environment

- Virtual environments give each project its own place for packages:



Creating a Virtual Environment

- The following creates a directory called `.venv` containing the virtual environment.

```
python3 -m venv .venv
```

- Then the following activates it:

```
source .venv/bin/activate
```

- After activation, commands such as:
 - `python3`
 - `pip`
- refer to the versions inside `.venv`.

Python Virtual Environments

- You can investigate this with:

```
$ which python
```

- Before activation, you might see:

```
$ /usr/bin/python
```

- After activation:

```
$ /home/student/project/.venv/bin/python
```

What's actually happening?

- Activation modifies your shell's environment (most importantly, **\$PATH**) so that the virtual environment's `bin` directory comes **before** the directories containing the system Python.
- A Python virtual environment generally **doesn't contain a complete independent copy of Python**.
 - The `python` inside `.venv/bin` is commonly a **link to an existing Python interpreter**, rather than another full copy of Python.
 - The virtual environment mainly provides an isolated location for the project's installed packages.

Bitwise Practice

Image Filters

- In hexadecimal notation, an RGB color is represented as RRGGBB, where:
 - RR represents the red component (0x00 to 0xFF)
 - GG represents the green component (0x00 to 0xFF)
 - BB represents the blue component (0x00 to 0xFF)
- For example, the color 0x4CAF50 consists of:
 - Red: 0x4C
 - Green: 0xAF
 - Blue: 0x50

Image Filters

Many photo-editing applications provide warm filters that make an image appear warmer by increasing the intensity of red tones. Write a function that applies a simple warm filter to a color by increasing its red channel by 0x20. If adding 0x20 would cause the red channel to exceed 0xFF, the red channel should instead be set to 0xFF. The green and blue channels should remain unchanged. Your function should have the following prototype:

```
uint32_t warm_filter(uint32_t hex_color);
```

	Example 1	Example 2
hex_color	0x123456	0xF0ABCD
Original Red	0x12	0xF0
New Red	0x32	0xFF
Expected Output	0x323456	0xFFABCD

Image Filters

```
uint32_t warm_filter(uint32_t hex_color) {  
    // Step 1: Extract the red channel  
  
    // Step 2: Increase red, without going above 0xFF  
  
    // Step 3: Clear the original red channel  
  
    // Step 4: Insert the new red channel  
  
    return hex_color;  
}
```

Image Filters



```
uint32_t warm_filter(uint32_t hex_color) {  
    // Step 1: Extract the red channel  
    uint32_t red = (hex_color >> 16) & 0xFF;  
  
    // Step 2: Increase red, without going above 0xFF  
    if (red + 0x20 > 0xFF) {  
        red = 0xFF;  
    } else {  
        red = red + 0x20;  
    }  
  
    // Step 3: Clear the original red channel  
    hex_color = hex_color & ~(0xFF << 16);  
  
    // Step 4: Insert the new red channel  
    hex_color = hex_color | (red << 16);  
  
    return hex_color;  
}
```

Stack Diagrams

```

#include <stdint.h>
#include <stdio.h>

#define ARR_SIZE 5

void update(int32_t arr[], int64_t value) {
    value *= 2;
    for (int8_t i = 0; i < ARR_SIZE; i += 3) {
        arr[i] = value * 2;
    }
}

int main() {
    int64_t x = 5;
    int32_t a[ARR_SIZE] = {1, 2, 3, 4, 5};
    update(a, x);

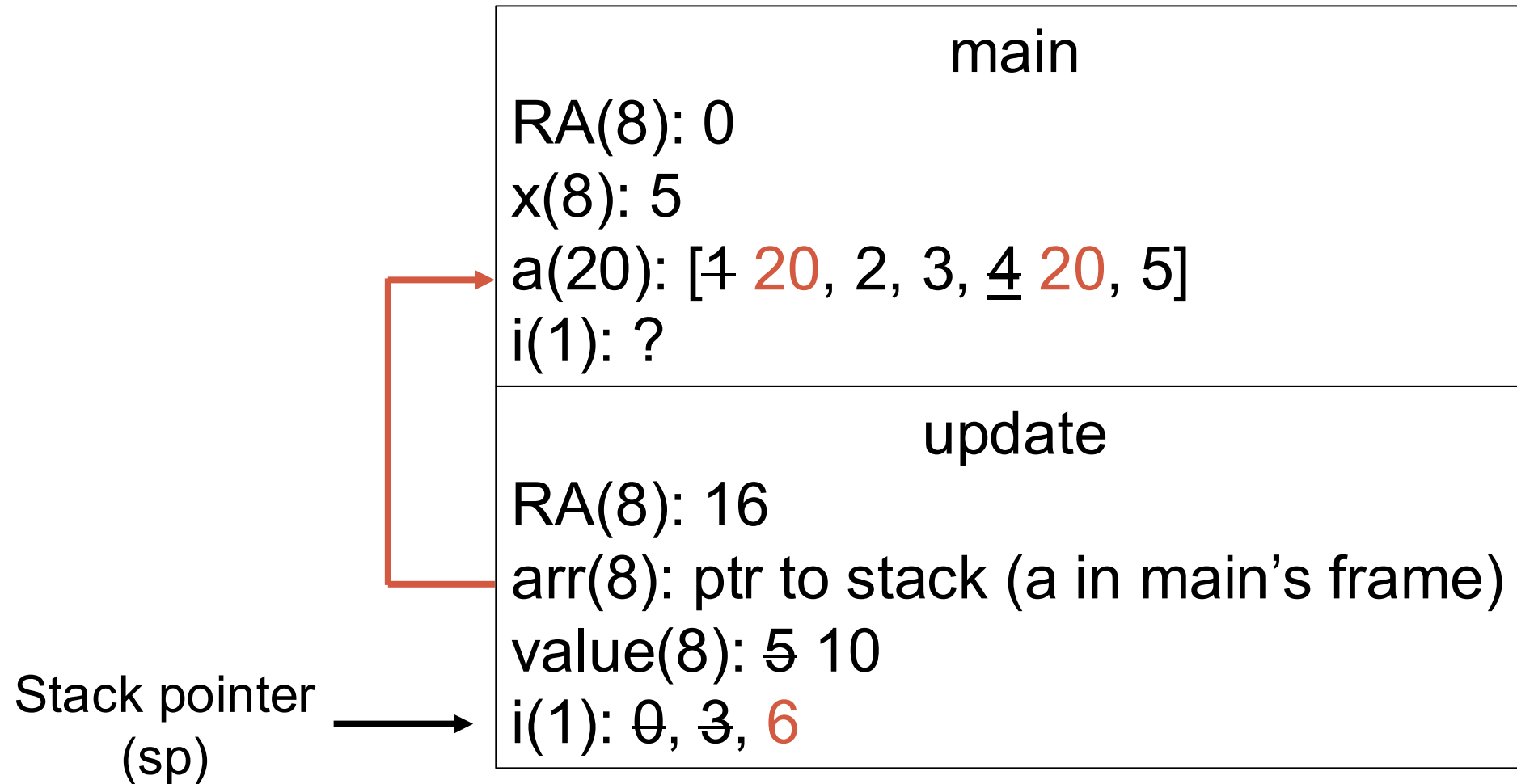
    printf("x: %ld\n", x);
    printf("sizeof(x): %lu\n", sizeof(x));
    printf("sizeof(a): %lu\n", sizeof(a));
    for (int8_t i = 0; i < ARR_SIZE; i++) {
        printf("%d: %d\n", i, a[i]);
    }
}

```

Draw the memory diagram for this program just before update returns.

- Indicate the location of the stack pointer just before update returns.

Draw the memory diagram for this program just before update returns.



What does the program output?

x: _____

sizeof(x): _____

sizeof(a): _____

0: _____

1: _____

2: _____

3: _____

4: _____

What does the program output?



```
x: 5
sizeof(x): 8
sizeof(a): 20
0: 20
1: 2
2: 3
3: 20
4: 5
```

Character Arrays

```

#include <stdio.h>
#include <string.h>

int main(void) {
    char str1[] = "211";
    char str2[] = "U\nN\nC\n";
    char str3[] = "She said \"hello\"!";
    char str4[25] = {'U', 'N', 'C', '\n', 't', 'a', '\0',
                    'r', 'h', 'e', 'e', 'l', 's', '\0'};

    char dest[20];
    strcpy(dest, str1);

    printf("str1: size = %lu, length = %lu\n\n", sizeof(str1), strlen(str1));
    printf("str2: size = %lu, length = %lu\n\n", sizeof(str2), strlen(str2));
    printf("str3: size = %lu, length = %lu\n\n", sizeof(str3), strlen(str3));
    printf("str4: size = %lu, length = %lu\n\n", sizeof(str4), strlen(str4));
    printf("dest: size = %lu, length = %lu\n\n", sizeof(dest), strlen(dest));

    printf("%s", str1);
    printf("%s", str2);
    printf("%s", str3);
    printf("%s", str4);

    printf("\nEnd\n");

    return 0;
}

```

Fill in the program output

str1: size = _____, length = _____

str2: size = _____, length = _____

str3: size = _____, length = _____

str4: size = _____, length = _____

dest: size = _____, length = _____

End

Fill in the program output

str1: size = 4, length = 3

str2: size = 7, length = 6

str3: size = 18, length = 17

str4: size = 25, length = 6

dest: size = 20, length = 3

211U

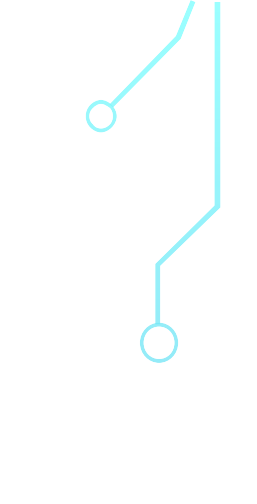
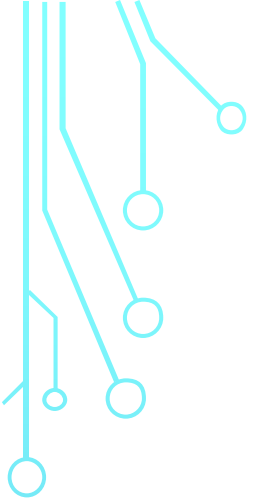
N

C

She said "hello"!UNC

ta

End



Glob

Glob

1. Write a command that removes all files in the current directory that start with `temp`.
2. Write a command that lists all files in the current working directory whose names contain at least one digit.
3. Write a find command that searches the current directory and all subdirectories for files ending in `.log`.
4. Write a command that lists all files whose names begin with either `a` or `b`, end with `.txt`, and may contain any number of any characters in between.
5. Write a command that copies all files in the current working directory whose names start with `data`, end with a digit, and may contain any number of any characters in between, into a subdirectory named `backup`.

Glob



1. Write a command that removes all files in the current directory that start with `temp`.
 - `rm temp*` OR `rm ./temp*`
2. Write a command that lists all files in the current working directory whose names contain at least one digit.
 - `ls *[0-9]*`
3. Write a find command that searches the current directory and all subdirectories for files ending in `.log`.
 - `find . -name "*.log"`
4. Write a command that lists all files whose names begin with either `a` or `b`, end with `.txt`, and may contain any number of any characters in between.
 - `ls [ab]*.txt` OR `ls a*.txt b*.txt`
5. Write a command that copies all files in the current working directory whose names start with `data`, end with a digit, and may contain any number of any characters in between, into a subdirectory named `backup`.
 - `cp data*[0-9] backup/`